UCS

Name : Asmit Sahu Roll : 23052231

In [1]:
```python
import heapq

def uniform_cost_search(graph, start, goal):
    # Priority queue: (cost, current_node, path)
    pq = [(0, start, [start])]
    visited = set()

    while pq:
        cost, node, path = heapq.heappop(pq)

        if node in visited:
            continue
        visited.add(node)

        if node == goal:
            return path, cost

        for neighbor, edge_cost in graph[node]:
            if neighbor not in visited:
                heapq.heappush(
                    pq,
                    (cost + edge_cost, neighbor, path + [neighbor])
                )

    return None, float("inf")
```

In [2]:
```python
graph = {
    'A': [('B', 2), ('C', 5)],
    'B': [('C', 1), ('D', 4)],
    'C': [('D', 1)],
    'D': []
}


path, cost = uniform_cost_search(graph, 'A', 'D')
print("Path:", path)
print("Total Cost:", cost)
```

```
Path: ['A', 'B', 'C', 'D']
Total Cost: 4
```

bfs compare

In [3]:
```python
from collections import deque

def bfs(graph, start, goal):
```

```
    queue = deque([(start, [start])])
    visited = set([start])

    while queue:
        node, path = queue.popleft()

        if node == goal:
            return path

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, path + [neighbor]))

    return None
```

In [5]:
```
graph_unweighted = {
    'A': ['B', 'C'],
    'B': ['C', 'D'],
    'C': ['D'],
    'D': []
}

bfs(graph_unweighted,"A","D")
```

Out[5]:
```
['A', 'B', 'D']
```

In [ ]:
```
'''

BFS we consider unweighted graph
so no cost consideration
dataStructure is queue but in ucs we use priority Queue a so it cosider as a min ma
TC(BFS) = O(V+E)
TC(UCS) = O(E logV)

'''
```